

Manual de Arquitectura y Desarrollo

Especificación de arquitectura, modelo de dominio, contratos de API y procedimientos técnicos

Código del Documento:	CREAPP-DEV-01
Versión:	2.0 (Desarrollo y Mantenimiento)
Autor:	Juan Martín Morales
Repositorio:	https://github.com/juanm4morales/cre-app
Clasificación:	Documentación Técnica de Ingeniería
Destinatarios:	Desarrolladores de Software, Arquitectos y Control de Calidad
Fecha de Revisión:	30 de septiembre de 2026

Universidad Nacional de Cuyo — Sistema de Gestión Curricular y Equivalencias (CREApp)

Este documento describe el modelo relacional, las reglas de negocio, los contratos HTTP y las guías de contribución.

Cobertura documental: desarrollo de producto y destinos Azure o self-hosted. Este documento no verifica infraestructura ni despliegues en ejecución.

Contenido

Propósito y alcance	1
1. Estructura	1
2. Entorno local	1
3. Backend	1
4. Frontend	4
5. Agregar una funcionalidad	6
6. Antes de integrar	6
7. Referencias	6

Propósito y alcance

Autor: Juan Martín Morales **Repositorio:** github.com/juanm4morales/cre-app **Versión:** 3.0

Referencia técnica para mantener y extender la aplicación. La fuente de verdad es el código: ante cualquier duda, revise los archivos citados y sus pruebas.

El detalle de ramas, diferencias de routing, deuda técnica y evidencia de validación está en [APENDICES.md](#) y [validation/ACCEPTANCE_REPORT.md](#). Para publicar en un entorno real, consulte el [manual de despliegue](#).

1. Estructura

Ruta	Responsabilidad
backend/config/	Settings, URLs, API, middleware, WSGI
backend/accounts/	Perfil, roles, sesión, permisos y gestión de docentes
backend/academics/	Catálogo académico y configuración CRE
backend/planning/	Asignaciones, programas, unidades, calendario, actividades IP/TA
frontend/src/pages/	Pantallas por rol (admin/, docente/)
frontend/src/	Layout, CRUD admin, formularios, tablas
components/	
frontend/src/contexts/, hooks/, services/	Sesión cliente, selección, refresco, cliente HTTP
docs/manuales/	Estos manuales y el harness de aceptación

2. Entorno local

```
python3 -m venv .venv && . .venv/bin/activate
pip install -r requirements.txt
cp backend/.env.example backend/.env      # ajustar credenciales

cd backend && docker compose up -d          # PostgreSQL de desarrollo
python manage.py migrate
python manage.py createsuperuser
python manage.py runserver
```

En otra terminal:

```
cd frontend && npm ci && npm run dev      # http://localhost:5173
```

Variables: el backend lee `backend/.env`; el frontend solo lee `VITE_API_URL` y `VITE_STATIC_BASE`, y ambas son de **compilación**. En same-origin use `VITE_API_URL=/api` y `VITE_STATIC_BASE=/api/static/`.

El Compose del repositorio levanta **solo PostgreSQL**, no la aplicación. No es un despliegue de producción.

3. Backend

3.1 Modelos

App	Modelo	Notas
accounts	UserProfile	OneToOne con User, rol ADMIN/DOCENTE; se crea por señal
academics	ConfiguracionCRE	Fila única lógica; define horas por CRE
academics	UnidadAcademica	sigla única
academics	Carrera	codigo único, PROTECT desde la unidad

academics	PlanEstudio	ordenanza única; horas = créditos × horas por CRE
academics	EspacioCurricular	codigo único; horas totales = IP + TA
academics	PlanEstudioEC	Une plan y espacio; CASCADE hacia programas
academics	Competencia	Código único por plan; baja lógica con activo
planning	AsignacionDocente	Rango de vigencia con CheckConstraint
planning	Programa	Único por plan-espacio y año; texto de fundamentación y objetivos
planning	Unidad, UnidadCompetencia	Competencias validan el mismo plan del programa
planning	DiaClasePrograma	Regla semanal; inicio y fin juntos
planning	ClaseCalendario	Ocurrencia única por programa y fecha
planning	TipoActividad	Nombre único; dedicación IP o TA
planning	Actividad	M2M con unidades; clean() no se invoca en save()
planning	ActividadAjuste	Historial de reemplazos y extensiones de horas IP

Borrado: Programa, Unidad, Actividad, DiaClasePrograma y Competencia usan baja lógica. Usuario se desactiva con `is_active=False`. AsignacionDocente no tiene baja: su vigencia depende de las fechas.

3.2 Sesión, roles y permisos

La autenticación es por sesión de Django con DRF; no hay tokens. El rol activo vive en la sesión y puede cambiar con `POST /api/auth/role`.

Endpoint	Acceso	Devuelve
GET /api/auth/csrf	Público	Cookie CSRF y token JSON
POST /api/auth/login	Público	Sesión iniciada, rol activo, token CSRF
GET /api/auth/me	Autenticado	Usuario, rol, roles disponibles
POST /api/auth/role	Autenticado	Cambia el rol activo
POST /api/auth/logout	Autenticado	Cierra la sesión

`get_available_roles()` concede `admin` si el usuario es `staff`, superusuario o tiene perfil `ADMIN`, y `docente` si el perfil falta o es `DOCENTE`. Los permisos de escritura comparan el **rol activo de la sesión**, no solo el perfil.

El alcance de un docente son sus espacios con asignación vigente hoy, no la historia ni el año del programa.

3.3 API

El router no usa barra final: `/api/programas`, no `/api/programas/`. Los listados usan paginación de 100 y responden `{count, next, previous, results}`, salvo `espacios-asignados`, que devuelve un array simple.

Recurso	Filtros de lista	Reglas
/api/unidades-academicas	—	Escritura solo admin
/api/carreras	unidad_academica_id	Escritura solo admin

/api/planes-estudio	carrera_id		Valida rango de vigencia
/api/planes-estudio-ec	plan_estudio_id, curricular_id	espacio_	No se desvincula si tiene programas
/api/espacios-curriculares	—		Escritura solo admin
/api/competencias	plan_estudio_id, include_inactive=1		Borrado lógico
/api/configuracion-cre	—		Fila única
/api/usuarios	—		Solo docentes; desactivar en vez de borrar
/api/asignaciones-docentes	docente_id, espacio_curricular_id		Rechaza usuarios no docentes y solapamientos
/api/tipos-actividad	—		Lectura autenticada, escritura admin
/api/programas	plan_estudio_ec_id		Ámbito por asignación; borrado lógico en cascada
/api/unidades	programa_id, plan_estudio_ec_id		Ámbito por asignación
/api/dias-clase	programa_id, plan_estudio_ec_id		Inicio y fin juntos
/api/actividades	programa_id, plan_estudio_ec_id		Validación distinta para IP y TA
/api/clases-calendario	programa_id, plan_estudio_ec_id, desde, hasta		Ámbito por asignación
/api/espacios-asignados	—		Admin ve todo; docente, lo asignado hoy

Acciones propias:

Método y ruta	Uso
POST /api/espacios-asignados/create_programa_if_needed	Crea el programa del año en curso; copia del anterior al crearlo
POST /api/espacios-asignados/temporal/espacio	Alta temporal de espacio para docentes (provisional)
PATCH /api/espacios-asignados/temporal/carga-horaria	Recalcula créditos desde las horas
POST /api/espacios-asignados/temporal/asignarme	Autovinculación de espacio existente (provisional)
POST /api/clases-calendario/generar-rango	Genera ocurrencias desde las reglas semanales
POST /api/unidades/{id}/competencias	Reemplaza el conjunto de competencias
GET/POST /api/actividades/{id}/ajustes	Historial de ajustes de una actividad

docs/API.md está desactualizado: usa barra final, nombres de recursos que no existen y filtros no implementados. Use el router como referencia.

3.4 Validación

`Model.save()` no llama a `full_clean()`. Las reglas implementadas en `clean()` o en serializers solo se aplican si quien escribe las invoca. Varias validaciones viven en el serializer y no están en la base, por lo que importaciones, scripts o accesos directos por ORM pueden evitarlas.

Al agregar una restricción que deba cumplirse siempre, impleméntela en el modelo y en la base con `CheckConstraint` o `UniqueConstraint`, no solo en el serializer.

3.5 Migraciones e importaciones

```
cd backend
python manage.py makemigrations
python manage.py makemigrations --check --dry-run
python manage.py migrate
python manage.py showmigrations
```

- `0011_seed_ip_tipo_actividad` crea los tipos IP que faltan; no cambia la dedicación de los existentes y su reversión no los borra.
- `0012_programa_competencias_programa_fundamentacion_and_more` agrega cuatro campos de texto a Programa: `fundamentacion`, `objetivos_generales`, `objetivos_especificos` y `competencias`. No agrega relaciones de competencias.

Importadores disponibles:

```
python manage.py import_academic_xlsx datos.xlsx \
--unidad-sigla FCE --unidad-nombre 'Facultad de Ciencias Económicas' \
--career-level G --plan-credits 300 --vigente-desde 2024-01-01 --dry-run
```

```
python manage.py import_tipo_actividad_xlsx tipos.xlsx --tipo-dedicacion IP --dry-run
```

`import_academic_xlsx` lee la hoja `Propuestas` y opcionalmente `Espacios`, `Competencias` y `Docentes`. Los docentes creados quedan con contraseña inutilizable: hay que establecerles credenciales antes de que puedan entrar. `import_tipo_actividad_xlsx` lee la primera hoja desde la fila 3 y rechaza un nombre cuya dedicación difiera, en vez de sobrescribirla.

3.6 Pruebas

```
cd backend
python manage.py test
python manage.py test accounts academics planning
```

Requieren PostgreSQL con permiso de creación de base. SQLite no reproduce las restricciones reales. La suite cubre sesión y roles, permisos, invariantes de modelos, importación, generación de calendario y flujos de API. Hay brechas de seguridad conocidas y pruebas que nombran problemas ya corregidos; revívelas antes de confiar en su nombre.

4. Frontend

4.1 Comandos

```
cd frontend
npm ci
npm run dev
npm run typecheck
npm test
npm run build
```

No hay script `lint`, aunque ESLint y sus plugins estén instalados.

4.2 Rutas

Ruta	Pantalla
<code>/login</code>	Inicio de sesión
<code>/docente/resumen</code>	Resumen docente
<code>/docente/espacios</code>	Espacios asignados y flujos temporales
<code>/docente/programas</code>	Programas, unidades y competencias
<code>/docente/agenda-cursado</code>	Agenda semanal
<code>/docente/planificacion-ip</code>	Planificación de interacción pedagógica
<code>/docente/planificacion-ta</code>	Planificación de trabajo autónomo
<code>/docente/ejecucion-ip</code>	Ejecución y seguimiento
<code>/docente/perfil</code>	Perfil del docente
<code>/admin/*</code>	Dashboards y catálogos administrativos
<code>/sadmin-creapp-panel</code>	Redirige al admin de Django en <code>/api/sadmin-creapp-panel/</code>

`ProtectedRoute` solo oculta pantallas: la autorización real está en el backend.

4.3 Sesión y datos

- `AuthContext` guarda el usuario en `localStorage` para mostrar la interfaz, pero la sesión se valida contra el backend al cargar.
- `services/api.ts` usa `VITE_API_URL || '/api'`, envía cookies, adjunta `X-CSRFToken` y ante errores de autenticación redirige al login.
- Las mutaciones emiten un evento global que dispara el refresco automático; las pantallas con `React Query` además invalidan sus claves explícitamente.
- `useDocenteSelection` guarda el espacio y plan seleccionados en `sessionStorage`; IP y TA dependen de esa selección.
- Los tipos de respuesta están declarados en cada pantalla, sin un modelo compartido. Al cambiar un serializer, actualice todos los consumidores.

Los formularios de IP y TA capturan minutos y convierten a horas decimales antes de enviar. Mantenga esa conversión.

4.4 Componentes reutilizables

- `components/Admin/AdminCrudPage.tsx`: CRUD configurable por endpoint, campos, columnas y payload. Acepta respuestas en array o paginadas. Para un catálogo nuevo suele bastar una página fina que lo configure.
- `components/Forms/PlanningActivityCommonFields.tsx`: campos compartidos de actividades.
- `components/Forms/PlanningCalendar.tsx`: calendario mensual con navegación por teclado.
- `components/Tables/BasicTable.tsx`: tabla con paginación en cliente.

El tema alterna `light/dark` sobre `<html>`, se persiste en `localStorage` como `cre_theme` y se inicializa en `index.html` para evitar el parpadeo. Los tokens de color están en `src/index.css`.

4.5 Pruebas

Vitest con `jsdom`. Los archivos en `src/test/` cubren contexto de autenticación, rutas protegidas, login, calendario, tablas, catálogos y los flujos de planificación. Usan `vi.mock` del cliente HTTP; limpie sesión, almacenamiento y caché de consultas entre pruebas.

5. Agregar una funcionalidad

Catálogo nuevo:

1. Defina el modelo, su unicidad, su `on_delete` y su política de baja en la app que corresponda.
2. Genere y revise la migración; si requiere datos iniciales, hágalos idempotentes.
3. Escriba el serializer con los campos de API y su validación.
4. Cree el viewset con permisos, filtros, orden y el queryset con `select_related` o `prefetch_related` donde corresponda.
5. Registre la ruta en `backend/config/api_urls.py` respetando la ausencia de barra final.
6. Cree la página en `frontend/src/pages/admin/`, regístrela en `App.tsx` y agregue el enlace en la navegación.
7. Agregue pruebas de modelo, serializer y API, incluyendo permisos y códigos de estado.

Acción nueva sobre un recurso:

1. Declárela con `@action` en el viewset y elija `detail=True` si depende de un objeto existente.
2. No confíe en un identificador recibido: cargue el objeto aplicando el mismo ámbito que usa el queryset.
3. Valide la entrada con un serializer dedicado y use `transaction.atomic` si escribe varias filas.
4. Devuelva un código de estado coherente y pruebe los casos sin permiso, con datos inválidos y duplicados.

6. Antes de integrar

- ☐ Revisé modelo, serializer, viewset, router y todas las pantallas que consumen el payload.
- ☐ Definí quién puede leer y escribir, y validé las relaciones en el servidor.
- ☐ Decidí borrado físico, lógico o por vigencia, y el `on_delete` correspondiente.
- ☐ Agregué la migración y las pruebas de permisos, alcance e integridad.
- ☐ Verifiqué rutas sin barra final y el formato de respuesta (array o `results`).
- ☐ Invalidé las consultas afectadas en el frontend.
- ☐ Ejecuté `manage.py check, makemigrations --check --dry-run, manage.py test, npm run typecheck, npm test` y `npm run build`.
- ☐ Actualicé la documentación de API afectada y no incluí secretos.

7. Referencias

- [Django settings](#)
- [Django ORM: consultas diferidas](#)
- [Django testing](#)
- [Django REST Framework](#)
- [TanStack Query](#)
- [React Router](#)
- [Vite](#)
- [Vercel: prácticas de rendimiento en React](#)